Patrones estructurales

Hoja de ejercicios (Alumno)

Hoja de ejercicios: Patrones estructurales, *Facade*

Patrón *Facade*

Escenario

Se desarrolla una aplicación relacionada con una misión espacial. El objetivo es ejecutar una **secuencia de lanzamiento** de la nave.

Para ello, el sistema cuenta con varios subsistemas independientes:

- `FuelSystem`, encargado de comprobar el combustible.
- `NavigationSystem`, encargado de inicializar la ruta de lanzamiento.
- `EngineSystem`, encargado de arrancar los motores.

La operación que se desea realizar es *iniciar el lanzamiento de la misión*. Sin embargo, para conseguirlo, es necesario coordinar varios objetos y conocer el orden correcto de invocación de sus operaciones.

Proyecto proporcionado (versión sin patrón)

En el fichero `Facade_NoPattern.zip` se proporciona un proyecto Java (en el paquete `space`) en el que la aplicación realiza directamente toda la secuencia de lanzamiento.

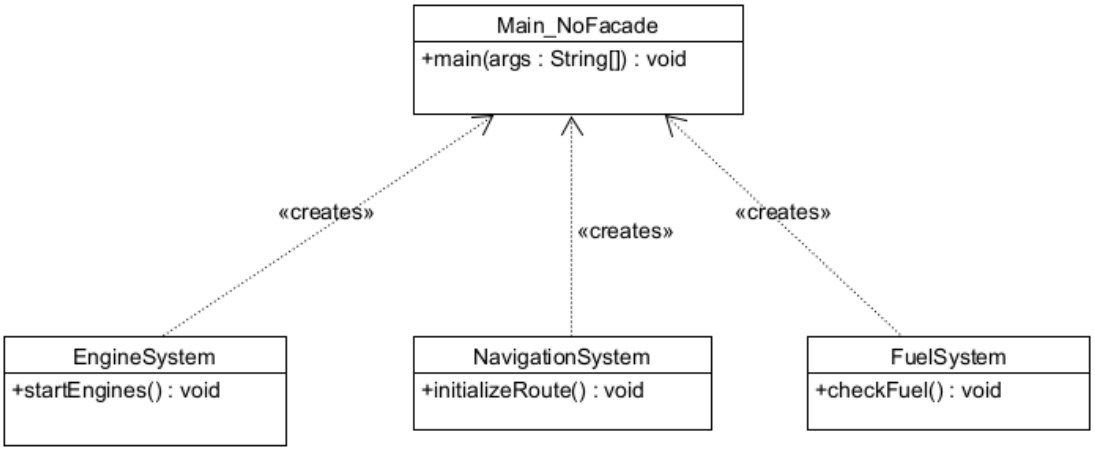
En esta versión sin patrón:

- la lógica de la aplicación en el programa principal crea explícitamente los subsistemas,
- conoce los nombres concretos de todas las clases implicadas,
- y coordina manualmente el orden de ejecución de las operaciones.

Esto hace que la lógica del programa principal quede excesivamente acoplada a la estructura interna de los subsistemas.

UML de la versión sin patrón

A continuación se te muestra el diagrama UML de esta solución sin patrón.



Se pide

A partir del proyecto proporcionado, debes desarrollar una nueva versión aplicando el patrón estructural *Facade*.

La solución con patrón deberá cumplir los siguientes requisitos:

- Los **subsistemas no deben modificarse**. En particular, deben mantenerse las clases:
  - `FuelSystem`
  - `NavigationSystem`
  - `EngineSystem`
- Debe añadirse una clase fachada llamada:
  - `MissionFacade`
- La fachada deberá encapsular la coordinación de los subsistemas y ofrecer una operación de alto nivel con la siguiente signatura:
  - `public void launchMission()`
- La fachada deberá contener internamente referencias a los tres subsistemas:
  - `FuelSystem`
  - `NavigationSystem`
  - `EngineSystem`
- Debe existir una clase principal de prueba:
  - `Main_Facade`

  que utilice la fachada en lugar de coordinar directamente los subsistemas.

Objetivo

Debes apreciar que el patrón *Facade* facilita :

- simplificar a la lógic de la aplicación el uso de un subsistema complejo, ya que la *Facade* en sí encapsula esa dificultad.
- reducir el acoplamiento de la lógica principal de la aplicación y los subsistemas del sistema complejo subyacente,
- y concentrar en una única clase la lógica de coordinación (la *Facade*).

Posible extensión

Considera ahora que se plantea otra tarea relacionada con la exploración espacial: el mantenimiento previo y posterior a una misión. Para ello ten en cuenta:

- Considera que cada uno de los sistemas pueda tener unos métodos adicionales para chequear que todo está bien o para reconfigurar el sistema.
- Considera la posibilidad de definir otra *Facade* (que no será jerárquicamente dependiente de la otra, es decir, no será una sub*Facade* que utlice la otra) para esta tarea.
- Observa también y compara el trabajo de reescritura más que de extensión si tuvieras que incorporar esa funcinalidad en la versión en la que no se usó el patrón.